APRA AMCOS Fuzzy Matching
Project Handover Documentation

Document prepared: 3 June 2025
Last updated: 6 June 2025
Version: 1.0

Created by: Shruti Sharma

shruti.sharma@billigence.com | +61 403916773

Table of Contents

1. Project Overview

This project implements a comprehensive fuzzy matching system between APRA works data and Muzooka tracks data. The system performs matching based on specified matching rules across four key dimensions: title matching, composer name matching, artist name matching and ISRC matching, utilizing Snowflake's native capabilities to identify potential matches between the two datasets.

The matching system follows a multi-stage approach:

1. **Title Matching**: Exact and fuzzy matching of track titles.
2. **Composer Matching**: Name-based matching of composers across datasets.
3. **Artist Matching**: Name-based matching of artists across datasets.
4. **ISRC Matching**: ISRC-based matching across datasets.

Technical Implementation
Deployed Snowflake Python notebooks with embedded JavaScript and SQL UDFs to deliver the implementation.

2. Title Matching Process

2.1 Data Preparation

- Title variants are generated and cleaned for both APRA Works and Muzooka tracks datasets.
- Cleaning rules are applied as per general title matching standards, handling bracketed and non-bracketed content variations.
- Title formats are standardized to improve matching accuracy.

2.2 Title Variant Generation Logic - Business Requirements

The implementation addresses all key business requirements as specified in the business requirements document, through a comprehensive JavaScript UDF (GENERATE_TITLE_VARIANTS_JS) that generates multiple title variations for matching.

2.2.1 Core Rule Compliance

Article Handling: The **removeArticles()** function removes leading and trailing articles (A, AN, THE) from all generated variants, ensuring clean comparisons as specified.

Bracket Normalization: The **cleanUnmatchedBrackets()** function handles unmatched brackets by:

- Removing orphaned opening brackets without corresponding closing brackets.
- Removing orphaned closing brackets without corresponding opening brackets.
- Preserving properly matched bracket pairs for further processing.

2.3 Methodology

2.3.1 Bracket Variation Logic

Single Bracket Groups: Creates variants both with and without bracketed content, supporting scenarios like:

- HELLO WORLD (NEW WORLD) → generates HELLO WORLD and HELLO WORLD NEW WORLD

Multiple Bracket Groups: Handles complex scenarios with leading/trailing brackets by generating up to 4 variants:

- Base text only (all brackets removed).
- First bracket group + base text.
- Base text + last bracket group.
- First bracket group + base text + last bracket group.

Middle Bracket Handling: For brackets appearing mid-title, generates variants both including and excluding the bracketed content while preserving surrounding text structure.

Complex Multi-Position Brackets: Implements the most sophisticated rule (Example 10 from Title matching section of business requirements document) by intelligently parsing bracket groups and generating all required combinations while accommodating rules for adjacent bracket sequences.

2.3.2 Output Structure for Variants

The function returns a sorted array of unique variants, eliminating duplicates and ensuring each title generates only meaningful, distinct variations for comprehensive fuzzy matching coverage.

This implementation provides the foundation for achieving the highest possible matching scores for Apra titles by testing all business-rule-compliant title variations against Muzooka titles.

2.3.3 Exact Title Matching

- Direct comparison between cleaned Muzooka track title variants and APRA work title variants.
- Exact matches stored with corresponding muzooka_track_id mappings.
- Results saved to ADC_WORKS_EXACT_MATCH_TITLE_VARIANTS table.

2.3.4 Fuzzy Title Matching

We are comparing all APRA work titles against Muzooka tracks that did not have an exact title match.

- Extracted 1 million rows for matching from ADC Works' **cleaned title variants,** to address computational overheads/ optimize performance.
- Utilized Snowflake's native **JAROWINKLER_SIMILARITY** function with similarity threshold ≥ 0.9.
- Matching optimization/Blocking technique: Matched on first three matching characters of titles between both datasets.
- Results stored in **FUZZY_TITLE_MATCHING** table.

2.3.5 Output Structure

- Union operation performed on exact and fuzzy matching results.
- Final consolidated dataset stored as **ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED.**

3. Composer Name Matching

3.1 Data Preparation

- Extracts composer data from both APRA and Muzooka composers' sources using work-track relationships established in title matching table ADC_WORKS_COLUMNS_TITLE_MATCHING_COMBINED.
- APRA composers joined by apra_work_id and Muzooka composers by muzooka_track_id.
- Composer names are pre-processed, and total composer counts are computed per work/track for detailed matching analysis.

3.2 Composer Matching Logic - Business Requirements

This implementation addresses client business requirements through a sophisticated part-by-part matching system with use case specific name parsing and scoring algorithms.

3.2.1 Core Rule Compliance

Individual Name Matching: The **create_composer_part_by_part_match_table()** function implements true individual composer matching by:

- Exploding delimited composer parts into individual components.
- Creating all possible combinations of APRA composer names × Muzooka tracks for a particular apra_work_id, for comprehensive comparison.
- Ensures each apra name is evaluated independently against all individual Muzooka composer names.

Name Format Recognition: The **parseNameComponents()** function handles both "First Last" and "Last First" formats by:

- Generating multiple format possibilities for each name.
- Testing all combinations: FIRSTNAME LASTNAME, LASTNAME FIRSTNAME, and compound variations.
- Ensuring perfect matches regardless of input order.

3.2.2 Name Processing

Last Name Priority: The **calculateMatchScore()** function implements strict last name matching by:

- Requiring minimum 80% last name similarity before considering other factors.
- Heavily weighting last name scores (60-75% of total score).
- Capping overall scores to a low similarity score when last names don't match well.

Middle Name Handling: Names are parsed to separate first, middle, and last components, with middle names being de-emphasized in scoring.

Generational Suffix Preservation: The **normalizeNameForComparison()** function correctly handles suffixes by:

- Recognizing SENIOR, SNR, SR, JUNIOR, JNR, JR as valid name components.
- Preserving these terms during normalization rather than discarding them.
- Including them in the matching algorithm for improved accuracy.

Initial Matching Logic: Dedicated name initial handling that matches single initials against full first names when last names are highly similar.

3.2.3 Scoring and Thresholds

0.92 Threshold Application: The system filters match using WHERE PART_MATCH_SCORE >= 0.92, ensuring only high-confidence matches are considered for final scoring.

Maximum Score Assignment: Following the business rule;

"The final name_match_score = highest score among individual part scores ≥ 0.92", this implementation uses MAX(PART_MATCH_SCORE) rather than averaging, giving each composer their best possible match score.

Multi-Algorithm Scoring: Combines Jaro-Winkler similarity, Levenshtein distance, and format-specific matching to provide comprehensive name comparison coverage.

3.2.4 Composer Match Percentage Calculation

Purpose: Calculates the percentage of composers successfully matched at the work-track level to measure matching coverage and quality.

Methodology: Sums all high-confidence match scores (≥ 0.92) using each composer's maximum part score rather than averaged scores and divides it by the maximum out of (Total Apra composers, Total Muzooka composers).

Output: COMPOSER_MTCH_PCNTG represents the proportion of composers with successful matches, where a perfect match of all composers would yield 1.0 (100%) and partial matching yields fractional values based on the cumulative match quality.

3.3 Methodology

3.3.1 Sample Processing

- Representative sample of apra_work_id records extracted from ADC_WORKS_TRACKS_MATCHED_COMPOSERS.
- Sample data stored in ADC_COMPOSERS_INTERMEDIATE table for detailed analysis.

3.3.2 Data Preprocessing

- APRA composer names cleaned and standardized.
- Composer name part counts calculated for granular matching.
- Total APRA composer count computed per apra_work_id.
- Muzooka composer data similarly cleaned and pre-processed.
- Total Muzooka composer count computed per muzooka_track_id.

3.3.3 Match Calculation

- Composer match percentages calculated for each APRA work.
- Each APRA composer name compared against all associated Muzooka tracks composers.
- Part-by-part matching algorithm implemented for comprehensive comparison.
- Results stored in COMPOSER_PART_BY_PART_MATCHING_FULL.

3.3.4 Output Structure

The final table provides composer-level matching with detailed metrics including individual part scores, total matched parts count, and work-track level composer match percentages, enabling comprehensive analysis of matching quality and coverage.

4. Artist Name Matching

4.1 Data Preparation

- Built upon ADC_COMPOSER_INTERMEDIATE as the base dataset.
- Artist data generated by matching from ADCARTISTS based on associated apra_work_id, including all associated muzooka_track_ids.
- Results saved as ADC_ARTISTS_INTERMEDIATE.
- APRA artist names cleaned and pre-processed following established patterns.

4.2 Artist Matching Logic - Business Requirements

This implementation addresses business requirements through a sophisticated part-by-part matching system that builds upon the same foundational algorithms as composer matching while incorporating artist-specific processing rules:

4.2.1 Core Architecture

Like the composer matching system, the artist implementation uses the same **create_enhanced_udf()** function with identical **parseNameComponents**(), **calculateMatchScore(),** and **normalizeNameForComparison()** algorithms, ensuring consistent matching behaviour across both entity types.

4.2.2 Artist-Specific Rule Implementation

Individual Artist Matching: The **create_part_by_part_match_table()** function implements comprehensive individual artist matching by:

- Exploding delimited artist parts into individual components on both APRA and Muzooka delimiter-separated names.
- Creating all possible combinations of APRA artist name parts × Muzooka artist name parts for exhaustive comparison.
- Ensuring each APRA artist name is evaluated independently against all individual Muzooka artist names.

Delimiter-Based Name Separation: Following the business requirement for "/" delimiter handling, the system:

- Processes delimiter-separated artist names stored in DELIMITED_PARTS columns from preprocessing tables
- Handles complex artist name structures like "Artist A / Artist B & Artist C" through systematic part explosion

4.2.3 Scoring and Thresholds

0.92 Threshold Application: Consistent with composer matching, the system applies WHERE PART_MATCH_SCORE >= 0.92 filtering to ensure only high-confidence artist matches contribute to final scoring.

Maximum Score Assignment: Implements MAX(PART_MATCH_SCORE) rather than averaging, ensuring each artist receives their highest possible match score among qualifying individual part comparisons.

Artist Match Total Calculation: The implementation includes work-track level aggregation (ARTIST_MATCH_TOTAL) that sums all qualifying artist match scores ≥ 0.92 providing comprehensive coverage metrics for multi-artist tracks.

Name Format Handling: Utilizes the similar name processing capabilities as composer matching, handling last name priority, multi-format recognition for names, etc.

4.3 Methodology

4.3.1 Match Scoring

- Artist match scores calculated for each APRA work.
- Each APRA artist name compared against corresponding Muzooka track artists.
- Part-by-part matching methodology applied consistently.
- Results stored in ARTIST_PART_BY_PART_MATCHING_FULL.

4.3.2 Output Structure

The final table provides artist-level matching with metrics including:

- Individual part match scores for each APRA artist vs. MUZOOKA artist comparison
- Artist count metadata (ADC_TOTAL_ARTISTS, MZK_TOTAL_ARTISTS) enabling coverage analysis
- Work-track level artist match totals for comprehensive matching assessment

5. ISRC Matching

Purpose: Identifies exact ISRC matches between APRA works and Muzooka tracks

5.1 Data Sources

- ADCISRC data is joined with ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED on matching apra_work_id along with their associated muzooka_track_id and then with Muzooka Recordings data based on established Muzooka track relationships.

5.2 Methodology

5.2.1 ISRC Normalization Process

- Removes hyphens and leading/trailing spaces from both APRA and Muzooka ISRC codes
- Truncates to 12 characters to handle formatting variations

5.2.2 Matching Logic

- Applies exact string matching between cleaned ISRC codes (CLEANED_R_ISRC = CLEANED_I_ISRC)
- Generates YN_ISRC_MATCH flag as a match indicator with 'Y' for successful matches, 'N' for non-matches

5.2.3 Output Structure

Creates ISRC_WITH_WORKS_TRACKS table containing only confirmed ISRC matches, providing high-confidence work-track associations

6. Key Performance Considerations

6.1 Scalability Constraints

- Cross-join operations limited to 1M rows for performance optimization
- Sampling strategies implemented to manage computational complexity
- Future scaling may require distributed processing or enhanced hardware resources

6.2 Similarity Thresholds

- Standardized threshold of 0.90 applied across title matching processes using JAROWINKLER_SIMILARITY.
- Threshold of 0.92 applied across artist, composer name matching processes.
- Threshold selected to balance precision and achieve performance optimization for matching results generation.
- Adjustable parameter for future tuning based on business requirements.

7. Database Schema

7.1 Primary Result Tables

Step	Table Name	Description
Title Matching	ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED	Comprises of UNION of FUZZY_TITLE_MATCHING and ADC_WORKS_EXACT_MATCH_TITLE_VARIANTS
Composer Matching	COMPOSER_PART_BY_PART_MATCH_FULL	Comprises of final match results along with writer match percentages and total composer name counts for both datasets.
Artist matching	ARTIST_PART_BY_PART_MATCH_FULL	Comprises of final match results along with artist match scores and total artist name counts for both datasets.
Consolidated Results	ADC_WORKS_ARTISTS_COMPOSERS_MATCHED	Comprises of joined results from works, composers and artist matching.
ISRC Matching	ISRC_WITH_WORKS_TRACKS	Comprises of ISRC match flag with Muzooka ISRCs for apra_work_ids from ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED

7.2 Intermediate Tables

Multiple intermediate tables are created at various stages to optimize performance, enhance Snowflake's query efficiency, and minimize execution time by addressing potential performance bottlenecks.

7.2.1 Title Matching

Table Name	Description
ADC_WORKS_TITLE_VARIANTS_CLEAN	Comprises of clean and separated Apra works title variants.
MZK_TRACKS_TITLE_VARIANTS_CLEAN	Comprises of clean and separated Muzooka tracks title variants.
ADC_WORKS_EXACT_MATCH_TITLE_VARIANTS	Comprises of exact Apra works title matches with Muzooka track titles, along with matching muzooka_track_id.

FUZZY_TITLE_MATCHING	Comprises of fuzzy Apra works title matches with Muzooka track titles, along with matching muzooka_track_id.
ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED	Comprises of UNION of FUZZY_TITLE_MATCHING and ADC_WORKS_EXACT_MATCH_TITLE_VARIANTS

7.2.2 Composer Matching

Table Name	Description
ADC_WORKS_TRACKS_MATCHED_COMPOSERS	Comprises of matching APRA composer data based on apra_work_id along with associated muzooka_track_id as obtained from ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED.
MZK_TRACKS_WORKS_MATCHED_COMPOSERS	Comprises of matching Muzooka composer data based on track ids as obtained from ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED.
ADC_COMPOSERS_INTERMEDIATE	Comprises of APRA composers' data for about 1100 unique apra_work_id, along with the associated muzooka_track_id.
ADC_COMPOSERS_DELIMITER_COUNT_CLEAN	Pre-processed APRA composer data with muzooka_track_id and composer counts that feeds into the composer's name matching logic.
MZK_COMPOSERS_DELIMITER_COUNT_CLEAN	Pre-processed Muzooka composer data and composer counts that feeds into the composer's name matching logic.

7.2.3 Artist Matching

Table Name	Description
ADC_ARTISTS_INTERMEDIATE	Comprises of APRA artists data matching on apra_work_id from ADC_COMPOSERS_INTERMEDIATE, along with the associated muzooka_track_id.
MZK_TRACKS_MATCHED_ARTISTS	Comprises of Muzooka artist data matching on track ids as obtained from ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED.

ADC_ARTISTS_DELIMITER_COUNT_CLEAN_	Comprises of pre-processed and cleaned APRA artists data along with muzooka_track_id, total artist name counts and individual name part counts.
MZK_ARTISTS_DELIMITER_COUNT_CLEAN	Comprises of pre-processed and cleaned Muzooka recordings data along with total artist name counts and individual name part counts.

7.2.4 Consolidated Table

Table Name	Description
ADC_WORKS_ARTISTS_COMPOSERS_MATCHED	Comprises of joined results from title, composers and artist matching.

7.2.5 ISRC Matching

Table Name	Description
ISRC_WITH_WORKS_TRACKS	Comprises of ISRC match flag to identify matching/non-matching ISRC ids amongst Muzooka Recordings and APRA Works data.

7.2 RDC Tables

A DELETE FROM statement has been added before each INSERT INTO script for the RDC tables.
This can be commented out if the data needs to be appended to the existing table.
However, it should be executed when only the new data is intended to remain in the table.

Table Name	Description
RDC_WORKS	Based on the results from intermediate table ADC_WORKS_ARTISTS_COMPOSERS_MATCHED
RDC_COMPOSERS	Based on the results from intermediate table COMPOSER_PART_BY_PART_MATCH_FULL
RDC_ARTISTS	Based on the results from intermediate table ARTIST_PART_BY_PART_MATCHING_FULL
RDC_ISRC	Based on the results from intermediate table ISRC_WITH_WORKS_TRACKS

8. Code Execution Guide

8.1 Python Notebooks

The Python notebooks to be run as advised against different matching steps sequentially.

Sno	Section	Python Notebook to Run	Tables Created
1	Title Matching	TITLE_VARIANT_AND_TITLE_MATCHING_COMPLETE	• ADC_WORKS_TITLE_VARIANTS • MZK_TRACKS_TITLE_VARIANTS • ADC_WORKS_TITLE_VARIANTS_CLEAN • MZK_TRACKS_TITLE_VARIANTS_CLEAN • ADC_WORKS_EXACT_MATCH_TITLE_VARIANTS • FUZZY_TITLE_MATCHING • ADC_WORKS_COLUMNS_TITLE_MATCHES_COMBINED
2	Composer Matching	COMPOSER_MATCHING_COMPLETE	• ADC_WORKS_TRACKS_MATCHED_COMPOSERS • MZK_TRACKS_WORKS_MATCHED_COMPOSERS • ADC_COMPOSERS_INTERMEDIATE • ADC_COMPOSERS_EXPANDED_CLEAN • MZK_COMPOSERS_EXPANDED_CLEAN • ADC_COMPOSER_DELIMITER_COUNT_CLEAN • MZK_COMPOSER_DELIMITER_COUNT_CLEAN • COMPOSERS_PART_BY_PART_MATCH_FULL
3	Artist Matching	ARTIST_MATCHING_COMPLETE	• ADC_ARTISTS_INTERMEDIATE • MZK_TRACKS_MATCHED_ARTISTS • ADC_APRA_ARTISTS_EXPANDED_CLEAN • MZK_TRACKS_ARTISTS_EXPANDED_CLEAN • ADC_ARTIST_DELIMITER_COUNT_CLEAN • MZK_ARTIST_DELIMITER_COUNT_CLEAN • ARTISTS_PART_BY_PART_MATCH_FULL
4	ISRC Matching	ISRC_MATCHING	• ISRC_WITH_WORKS_TRACKS
5	RDC Tables	RDC_TABLES	• ADC_WORKS_ARTISTS_COMPOSERS_MATCHED • RDC_WORKS • RDC_COMPOSERS • RDC_ARTISTS • RDC_ISRC

8.2 Full Dataset Run Instructions

We are applying sampling at two stages during the entire matching process:

1. At the FUZZY_TITLE_MATCHING stage

2. During the creation of the ADC_COMPOSERS_INTERMEDIATE table

To execute the matching process on the full dataset, please update the two specified code snippets accordingly.

Sno	Section	Python Notebook	Description
1	Title Matching	TITLE_VARIANT_AND_TITLE_MATCHING_COMPLETE	Modify the SQL Code for **FUZZY_TITLE_MATCHING** table creation, comment this line within cte1 for full dataset run, change **LIMIT 1000000** to **-- LIMIT 1000000**
2	Composer Matching	COMPOSER_MATCHING_COMPLETE	Modify the SQL Code for **ADC_COMPOSERS_INTERMEDIATE** table creation, change **SAMPLE (0.0002)** to **SAMPLE (100),** to process all apra_work_id (s) in the data, instead of a sample.

Relevant comments have also been added within the scripts for easier reference.

9. Maintenance and Support

9.1 Data Quality Monitoring

- Validation and updates on similarity thresholds, blocking techniques (currently 3-character matching for fuzzy title matching) and matching accuracies.
- Monitoring of processing performance and execution times.
- Quality and match score checks on cleaned and pre-processed data.

10. Future Scope

- Handling name transpositions, colliquations and synonyms in title/composer/artist name string matching.
- Handling UNICODE characters in names.
- Change sample percentages to operate on larger volumes with Snowpark optimized warehouses for improved execution times and querying capabilities.
- Consider implementing machine learning models for improved matching accuracy.
- Implement automated data pipeline monitoring and alerting in case the data needs to be streamed.